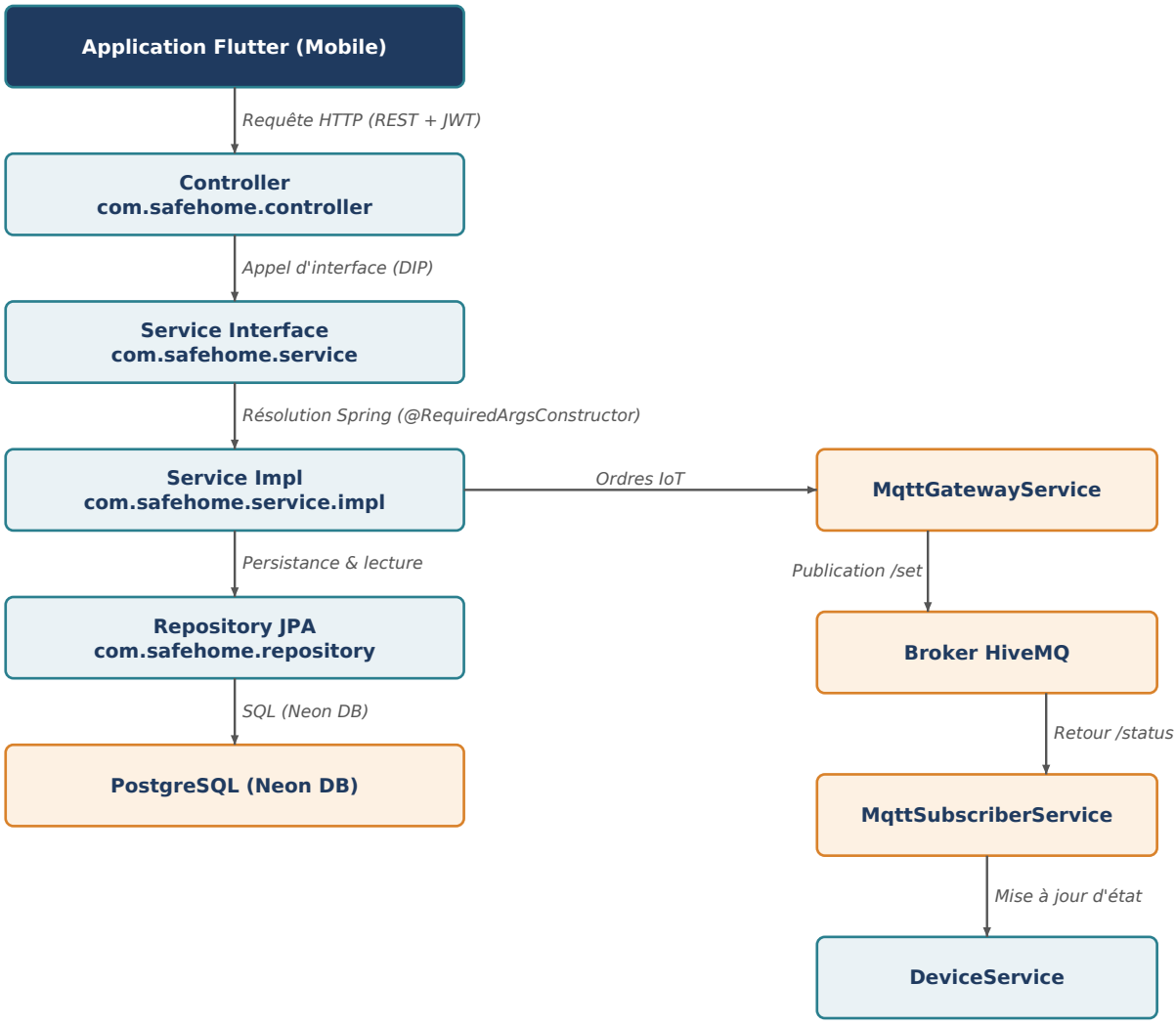
SafeHome

Architecture du backend : couches, fonctionnalités et passation de relais

Cette note explique l'architecture du backend SafeHome, la manière dont les fonctionnalités s'articulent et la passation de relais entre les différentes couches (Controller → Service → Repository / MQTT → Database).

1. Vue d'ensemble de la passation de relais (flux global)

Le backend respecte une architecture en couches stricte, couplée au principe d'Inversion des Dépendances (DIP) de SOLID.



Les règles de passation

1. **Les Controllers ne parlent jamais à la base de données :** ils ne connaissent que les interfaces de services et les DTOs `record`.
2. **Les Services ne dépendent jamais des Controllers :** ils implémentent une interface métier et orchestrent les Repositories, le service MQTT et l'audit.
3. **Immutabilité des échanges :** toutes les entrées et sorties de l'API utilisent des `record` Java (données immuables, rapides et thread-safe).

2. Fonctionnalité 1 : Sécurité, authentification & traçabilité (Écran 4)

A. Le flux de connexion `POST /api/auth/login`

1. **AuthController :** reçoit le DTO record `LoginRequest` validé par `@Valid` (email et password non vides). Il appelle l'interface `AuthService.login(...)`.
2. **AuthServiceImpl :**

- Interroge `UserRepository` pour chercher l'entité `User`.
- Utilise `PasswordEncoder` (BCryptPasswordEncoder) pour vérifier que le mot de passe correspond au hash stocké en base.
- Appelle `AuditService` pour enregistrer la connexion dans la table `audit_logs`.
- Délègue à `JwtUtils` la génération d'un token signé HMAC-SHA contenant l'ID, l'email et le rôle `ROLE_USER`.
- Retourne le record `AuthResponse` (token JWT + profil).

B. La validation des requêtes sécurisées `JwtAuthenticationFilter`

- À chaque requête vers une route protégée (`/api/devices`, `/api/alerts`, etc.), `JwtAuthenticationFilter` intercepte le header `Authorization: Bearer <token>`.
- Il valide le token avec `JwtUtils`, extrait l'email, récupère le profil utilisateur et peuple le `SecurityContextHolder`.

C. Le journal d'audit `AuditLogController` & `AuditServiceImpl`

- Chaque action critique (connexion, commande d'un volet, alerte d'urgence, scénario) invoque `auditService.log(user, action, target)`.
- `AuditLogController` expose `GET /api/audit` pour afficher l'historique complet : qui a agi sur quel équipement et à quelle heure.

3. Fonctionnalité 2 : Gestion des équipements & passerelle MQTT (Écran 1)

A. Consultation des équipements `GET /api/devices`

1. Le client Flutter interroge `DeviceController` (globalement, par zone SALON, CUISINE, CHAMBRE, GARAGE, ou par type LIGHT, LOCK, etc.).
2. Le controller appelle `DeviceService` qui extrait les entités depuis `DeviceRepository`.
3. Les entités sont immédiatement projetées en records `DeviceResponse` via `DeviceResponse.fromEntity(...)` pour garantir qu'aucune entité JPA mutable ne transite sur le réseau.

B. Envoi d'une commande actionneur `POST /api/devices/{id}/command`

C'est ici que le principe SRP (Single Responsibility Principle) prend toute son importance :

1. `DeviceController` reçoit le record `DeviceCommandRequest` (ex. `{"command": "{\"state\": \"ON\"}"}`) et extrait l'email de l'utilisateur connecté via `Authentication`.
2. **DeviceServiceImpl :**

- Met à jour l'état `stateValue` du `Device` en BDD via `DeviceRepository`.
- Passe le relais à `MqttGatewayService` pour émettre la commande physique.
- Passe le relais à `AuditService` pour inscrire l'action au journal d'audit.

3. `MqttGatewayServiceImpl` formate le topic officiel `safehome_charo_x789/home/{zone}/{device}/set` et injecte le message dans le canal sortant `mqttOutboundChannel`.

C. Réception réactive des retours d'états MQTT

1. Lorsqu'un capteur réel ou le simulateur 3D publie un retour sur `safehome_charo_x789/home/salon/light/status`, l'adaptateur Spring Integration (`MqttPahoMessageDrivenChannelAdapter`) l'intercepte.
2. Le message arrive dans `MqttSubscriberService` via `@ServiceActivator`.
3. Il analyse le topic (zone = SALON, device = light), résout l'enum et appelle `deviceService.updateDeviceStatus(...)` pour actualiser la BDD sans bloquer le broker.

4. Fonctionnalité 3 : Sécurité, incidents & alertes en temps réel (Écran 2)

A. Mode démo / simulation d'incident `POST /api/alerts/simulate`

1. Reçoit `SimulateIncidentRequest` (ex. zone CUISINE, message « Fumée détectée »).
2. **AlertServiceImpl :**

- Crée et sauvegarde une entité `Alert` non résolue en base de données.
- Publie la simulation sur le topic MQTT du capteur concerné via `MqttGatewayService` (ex. `home/cuisine/smoke/set`).
- Journalise la simulation dans l'audit.

B. Bouton d'urgence général `POST /api/alerts/emergency`

En cas de danger pressant (intrusion ou incident majeur) :

1. `AlertServiceImpl` recherche tous les verrous et portes de garage (`DeviceType.LOCK`).
2. Passe leur statut à « LOCKED » en BDD et envoie l'ordre de fermeture immédiate via MQTT.
3. Publie l'alarme générale sur `safehome_charo_x789/home/security/alarm`.
4. Enregistre une alerte CRITICAL et une entrée d'audit prioritaire.

5. Fonctionnalité 4 : Scénarios automatiques & routines (Écran 3)

A. Scénarios 1-clic `POST /api/routines/scenarios/trigger`

`RoutineServiceImpl` orchestre plusieurs appareils simultanément selon le mode :

Mode DEPART_ABSENCE

- Éteint toutes les lampes de toutes les pièces.
- Verrouille le garage.
- Ferme le volet roulant de la chambre (position 0 %).
- Arme les détecteurs de présence.

Mode NUIT

- Éteint salon et cuisine.
- Verrouille la porte d'entrée et le garage.

Mode SECURITE_INCENDIE

- Déverrouille toutes les issues pour faciliter l'évacuation.
- Ouvre tous les volets à 100 % pour évacuer les fumées.

Chaque scénario met à jour la base de données, publie les ordres MQTT correspondants, notifie `home/scenarios/active` et inscrit l'événement dans le journal d'audit.

B. Routines programmées `@Scheduled`

- Activé par `@EnableScheduling` sur `SafehomeApiApplication`.
- La méthode `scheduledNightSecurityCheck()` s'exécute automatiquement tous les soirs à 22h00 (`@Scheduled(cron = "0 0 22 * * ?")`) pour activer le mode NUIT en tâche de fond, sans intervention humaine.

En résumé : pourquoi cette organisation est solide

- **Aucun couplage fort :** si le broker MQTT change d'adresse ou de protocole, seuls MqttConfig et MqttGatewayServiceImpl sont impactés ; les Controllers et les règles métier ne bougent pas.
- **Traçabilité totale :** chaque action déclenchée par un utilisateur ou une tâche automatique est horodatée et conservée dans l'audit.
- **Sécurité et performance :** les DTOs en record Java évitent les effets de bord de mutation d'état, et Spring Security assure l'étanchéité des endpoints par JWT.